snyk
API & Web

PCI DSS 3.2.1 COMPLIANCE REPORT

Crinava - https://crinava26.netlify.app/

Report generated on Jul. 18, 2026 at 12:25 UTC

Summary

This section contains the scan summary

TARGET https://crinava26.netlify.app/

Report generated on Jul. 18, 2026 at 12:25 UTC

STARTED	ENDED	DURATION	SCAN PROFILE
Jul. 18, 2026, 12:10 UTC	Jul. 18, 2026, 12:22 UTC	11 minutes	Normal

NUMBER OF FINDINGS

	CURRENT SCAN	FROM LAST SCAN	PENDING FIX
CRITICAL	0	–	0
HIGH	0	–	0
MEDIUM	1	–	1
LOW	5	–	5

TOP 5

Missing Content Security Policy header	1
Missing clickjacking protection	1
Secure TLS protocol version 1.2 not supported	1
Weak cipher suites enabled	1
Browser content sniffing allowed	1

PCI DSS 3.2.1 REQUIREMENTS CHECKLIST

	TESTED	PASSED
4.1 Use strong cryptography and security protocols to safeguard sensitive cardholder data during transmission over public networks	✓	✗
6.2 Ensure that all system components have the latest security patches installed	✓	✓
6.5.1 Address injection flaws (SQL injection, OS Command injection, XPath, etc)	✓	✓
6.5.4 Address insecure communication flaws	✓	✗
6.5.5 Address improper error handling flaws	✓	✓
6.5.6 Address all “high risk” identified vulnerabilities	✓	✓
6.5.7 Address Cross-site scripting (XSS) flaws	✓	✓
6.5.8 Address improper access control flaws	✓*	—
6.5.9 Address Cross-site request forgery (CSRF) flaws	✓	✓

6.5.10 Address Broken authentication and session management flaws	✓	✓
6.6 Ensure the application is tested for vulnerabilities in an automated way	✓*	—

* Partially tested

Settings

This section contains the summary of settings that were used during this scan

---

✓ **SCAN PROFILE**

Normal

Tests for all supported vulnerabilities. It's the recommended scanning profile since it offers the best quality/time ratio.

Technical Summary

The following table summarizes the findings, ordered by their severity

#	SEVERITY		VULNERABILITY	STATE
4	MEDIUM	NEW	**Weak cipher suites enabled** https://crinava26.netlify.app/	**NOT FIXED**
3	LOW	NEW	**Secure TLS protocol version 1.2 not supported** https://crinava26.netlify.app/	**NOT FIXED**
6	LOW	NEW	**Referrer policy not defined** https://crinava26.netlify.app/	**NOT FIXED**
1	LOW	NEW	**Missing Content Security Policy header** https://crinava26.netlify.app/	**NOT FIXED**
2	LOW	NEW	**Missing clickjacking protection** https://crinava26.netlify.app/	**NOT FIXED**
5	LOW	NEW	**Browser content sniffing allowed** https://crinava26.netlify.app/	**NOT FIXED**

Exhaustive Test List

The following pages contain the list of vulnerabilities we tested in this scan, taking into consideration the chosen profile

---

- Reflected cross-site scripting
- Cookie without HttpOnly flag
- Open redirection
- SQL Injection
- Missing cross-site request forgery protection
- Missing clickjacking protection
- Stored cross-site scripting
- Insecure crossdomain.xml policy
- SSL cookie without Secure flag
- HTTP TRACE method enabled
- Directory Listing
- ASP.NET tracing enabled
- Path traversal
- Remote File Inclusion
- ASP.NET ViewState without MAC
- Session Token in URL
- Application error message
- Private IP addresses disclosed
- OS command injection
- XML external entity injection
- ASP.NET debugging enabled
- Insecure Silverlight clientaccesspolicy.xml policy
- PHP code injection
- Server-side JavaScript injection
- Ruby code injection
- Python code injection
- Server-side template injection
- Unencrypted communications
- HSTS header not enforced
- Mixed content
- Cross Origin Resource Sharing: Arbitrary Origin Trusted
- Certificate with insufficient key size or usage, or insecure signature algorithm
- Expired TLS certificate
- Insecure SSL protocol version 3 supported
- Deprecated TLS protocol version 1.0 supported
- Deprecated TLS protocol version 1.1 supported
- Secure TLS protocol version 1.2 not supported
- Weak cipher suites enabled
- Server Cipher Order not configured
- Untrusted TLS certificate
- Heartbleed
- Secure Renegotiation is not supported
- TLS Downgrade attack prevention not supported
- Drupal version with known vulnerabilities
- WordPress version with known vulnerabilities
- Joomla! version with known vulnerabilities

- Certificate without revocation information
- Full path disclosure
- Log file disclosure
- Backup file disclosure
- HSTS header set in HTTP
- HSTS header with low duration and no subdomain protection
- HSTS header with low duration
- HSTS header does not protect subdomains
- Inclusion of cryptocurrency mining script
- Insecure SSL protocol version 2 supported
- Browser content sniffing allowed
- Referrer policy not defined
- Insecure referrer policy
- Potential DoS on TLS Client Renegotiation
- JQuery library with known vulnerabilities
- AngularJS library with known vulnerabilities
- Bootstrap library with known vulnerabilities
- JQuery Mobile library with known vulnerabilities
- JQuery Migrate library with known vulnerabilities
- TLS certificate about to expire
- Moment.js library with known vulnerabilities
- Prototype library with known vulnerabilities
- React library with known vulnerabilities
- SWFObject library with known vulnerabilities
- TinyMCE library with known vulnerabilities
- Backbone library with known vulnerabilities
- Mustache library with known vulnerabilities
- Handlebars library with known vulnerabilities
- Dojo library with known vulnerabilities
- jPlayer library with known vulnerabilities
- CKEditor library with known vulnerabilities
- DWR library with known vulnerabilities
- Flowplayer library with known vulnerabilities
- DOMPurify library with known vulnerabilities
- Plupload library with known vulnerabilities
- easyXDM library with known vulnerabilities
- Ember library with known vulnerabilities
- YUI library with known vulnerabilities
- Sessvars library with known vulnerabilities
- prettyPhoto library with known vulnerabilities
- jQuery UI library with known vulnerabilities
- WordPress plugin with known vulnerabilities
- Invalid referrer policy
- Insecure PHP Object deserialization
- Missing Content Security Policy header
- Insecure Content Security Policy
- GraphQL Introspection enabled
- Log4Shell
- Vue.js library with known vulnerabilities
- Spring Cloud SPEL Code Injection (CVE-2022-22963)
- Spring4Shell
- Knockout library with known vulnerabilities
- Weak JWT HMAC secret
- JWT accepting none algorithm

- JWT signature is not being verified
- Using jwk parameter to verify JWTs
- JWT algorithm confusion
- MongoDB Injection
- Next.js library with known vulnerabilities
- Underscore.js library with known vulnerabilities
- Chart.js library with known vulnerabilities
- JSZip library with known vulnerabilities
- GraphQL Misconfiguration
- Insecure browser XSS protection enabled
- Hidden file found
- Svelte library with known vulnerabilities
- Axios library with known vulnerabilities
- Cookie with SameSite attribute set to None
- Server-side request forgery
- CRLF injection
- Froala library with known vulnerabilities
- Highcharts library with known vulnerabilities
- Supply Chain Compromise
- PDF.js library with known vulnerabilities
- Lodash library with known vulnerabilities
- Select2 library with known vulnerabilities
- UAParser.js library with known vulnerabilities
- MathJax library with known vulnerabilities
- JQuery Validation library with known vulnerabilities
- Ext JS library with known vulnerabilities
- jQuery File Upload library with known vulnerabilities
- React2Shell
- markdown-it library with known vulnerabilities
- highlight.js library with known vulnerabilities

Detailed Finding Descriptions

This section contains the findings in more detail, ordered by severity

---

4 Weak cipher suites enabled

MEDIUM

CVSS SCORE
4.2

CVSS:3.0/AV:A/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N

PATH

https://crinava26.netlify.app/

DESCRIPTION

The server supports weak cipher suites for SSL/TLS connections. These cipher suites are currently considered broken and, dependin g on the specific cipher suite, offer poor or no security at all. Thus defeating the purpose of using a secure communication channel in the first place.

Any connection to the server using a weak cipher suite is at risk of being eavesdropped and tampered with by an attacker that ca n intercept connections. This is more likely to occur to Wi-Fi clients.

Depending on the cipher suites used, a connection may be at an immediate risk of being intercepted.

The following issues need to be addressed:

- No cipher suites supporting Forward Secrecy enabled

EVIDENCE

No evidence available.

3 Secure TLS protocol version 1.2 not supported

LOW

CVSS SCORE
4.2

CVSS:3.0/AV:A/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N

PATH

https://crinava26.netlify.app/

DESCRIPTION

TLS protocol version 1.2 is not enabled on your server. We strongly recommend that TLS 1.2 is enabled, as it is both safer and faster than previous versions and protocols.

One of the most important features of TLS 1.2 is that it supports AEAD ciphers (Authenticated Encryption with Associated Data). Some non-AEAD ciphers are vulnerable to a class of attacks called padding-oracles, which led to the publication of attacks such as Lucky Thirteen and POODLE, amongst others. Moreover, as of 2018, TLS 1.2 adoption is above 94%. By supporting AEAD ciphers, TLS 1.2 is much more robust against padding-oracle attacks.

Additionally, depending on CPU support, some AEAD ciphers are actually faster to execute than their older counterparts.

The following issues need to be addressed:

- Not offered

EVIDENCE

No evidence available.

6 Referrer policy not defined

LOW

CVSS SCORE
3.1 CVSS:3.0/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N

METHOD	PATH
GET	https://crinava26.netlify.app/

DESCRIPTION

The application does not prevent browsers from sending sensitive information to third party sites in the **referer** header.

Without a referrer policy, every time a user clicks a link that takes him to another origin (domain), the browser will add a **referer** header with the URL from which he is coming from. That URL may contain sensitive information, such as password recovery tokens or personal information, and it will be visible that other origin. For instance, if the user is at `example.com/password_recovery?unique_token=14f748d89d` and clicks a link to `example-analytics.com`, that origin will receive the complete password recovery URL in the headers and might be able to set the users password. The same happens for requests made automatically by the application, such as XHR ones.

Applications should set a secure referrer policy that prevents sensitive data from being sent to third party sites.

EVIDENCE

Response headers, missing the `Referrer-Policy` header:

```
accept-ranges: bytes
age: 24
cache-control: public,max-age=0,must-revalidate
cache-status: "Netlify Edge"; hit
content-encoding: br
content-type: text/html; charset=UTF-8
date: Sat, 18 Jul 2026 12:13:23 GMT
etag: "f77462680fa9f1b82dd11b51c12a8f36-ssl-df"
server: Netlify
strict-transport-security: max-age=31536000; includeSubDomains; preload
vary: Accept-Encoding
x-nf-request-id: 01KXTJAGFC6ANTWGE0J5GS1K4Y
content-length: 662
```

REQUEST

```
GET / HTTP/2
accept: */*
accept-encoding: gzip, deflate, br
connection: keep-alive
user-agent: Mozilla/5.0 (compatible; +https://probely.com/sos)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36
ProbelySPDR/0.2.0
host: crinava26.netlify.app
```

RESPONSE

```
HTTP/2 200 OK
accept-ranges: bytes
age: 24
cache-control: public,max-age=0,must-revalidate
cache-status: "Netlify Edge"; hit
content-encoding: br
content-type: text/html; charset=UTF-8
date: Sat, 18 Jul 2026 12:13:23 GMT
etag: "f77462680fa9f1b82dd11b51c12a8f36-ssl-df"
```

```
server: Netlify
strict-transport-security: max-age=31536000; includeSubDomains; preload
vary: Accept-Encoding
x-nf-request-id: 01KXTJAGFC6ANTWGE0J5GS1K4Y
content-length: 662
<!doctype html>
<html lang="en" class="dark">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Crinava — The New Era of Cricket</title>
    <meta name="description" content="Crinava: Next-generation cricket
analytics, live match intelligence, and AI-powered predictions." />
    <script>
      window.process = { env: {} };
    </script>
    <!-- Celestial Organic Typography -->
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link href="https://fonts.googleapis.com/css2?family=JetBrains+Mono:wght@300
;400;500&family=Playfair+Display:wght@700;900&family=Inter:wght@400;500;600;700;
800;900&display=swap" rel="stylesheet" />
    <link href="https://fonts.googleapis.com/css2?family=Material+Symbols+Outlin
ed:wght,FILL@100..700,0..1&display=swap" rel="stylesheet" />
    <meta name='probely-verification'
content='c5ce0764-3330-42f5-a022-8a86effd23df' />
```

1

Missing Content Security Policy header

LOW

CVSS SCORE
3.7

CVSS:3.0/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N

METHOD	PATH
GET	https://crinava26.netlify.app/

DESCRIPTION

The Content Security Policy (CSP) is an HTTP header through which site owners define a set of security rules that the browser mus t follow when rendering their site. The most common usage is to define a list of approved sources of content that the browser can load. This can be used to effectively mitigate Cross-Site Scripting (XSS) and Clickjacking attacks.

CSP is flexible enough for you to define from where the browser can load JavaScript, Stylesheets, images, or fonts, among other op tions. It can also be used in report mode only, a recommended approach before deploying strict rules in a live environment. Howe ver, please note that report mode does not protect you, it just logs policy violations.

EVIDENCE

Response headers, missing the Content-Security-Policy header:

```
accept-ranges: bytes
age: 24
cache-control: public,max-age=0,must-revalidate
cache-status: "Netlify Edge"; hit
content-encoding: br
content-type: text/html; charset=UTF-8
date: Sat, 18 Jul 2026 12:13:23 GMT
etag: "f77462680fa9f1b82dd11b51c12a8f36-ssl-df"
server: Netlify
strict-transport-security: max-age=31536000; includeSubDomains; preload
vary: Accept-Encoding
x-nf-request-id: 01KXTJAGFC6ANTWGE0J5GS1K4Y
content-length: 662
```

REQUEST

```
GET / HTTP/2
accept: */*
accept-encoding: gzip, deflate, br
connection: keep-alive
user-agent: Mozilla/5.0 (compatible; +https://probely.com/sos)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36
ProbelySPDR/0.2.0
host: crinava26.netlify.app
```

RESPONSE

```
HTTP/2 200 OK
accept-ranges: bytes
age: 24
cache-control: public,max-age=0,must-revalidate
cache-status: "Netlify Edge"; hit
content-encoding: br
content-type: text/html; charset=UTF-8
date: Sat, 18 Jul 2026 12:13:23 GMT
etag: "f77462680fa9f1b82dd11b51c12a8f36-ssl-df"
server: Netlify
strict-transport-security: max-age=31536000; includeSubDomains; preload
vary: Accept-Encoding
```

```
x-nf-request-id: 01KXTJAGFC6ANTWGE0J5GS1K4Y
content-length: 662
<!doctype html>
<html lang="en" class="dark">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Crinava — The New Era of Cricket</title>
    <meta name="description" content="Crinava: Next-generation cricket
analytics, live match intelligence, and AI-powered predictions." />
    <script>
      window.process = { env: {} };
    </script>
    <!-- Celestial Organic Typography -->
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link href="https://fonts.googleapis.com/css2?family=JetBrains+Mono:wght@300
;400;500&family=Playfair+Display:wght@700;900&family=Inter:wght@400;500;600;700;
800;900&display=swap" rel="stylesheet" />
    <link href="https://fonts.googleapis.com/css2?family=Material+Symbols+Outlin
ed:wght,FILL@100..700,0..1&display=swap" rel="stylesheet" />
    <meta name='probely-verification'
content='c5ce0764-3330-42f5-a022-8a86effd23df' />
```

2 Missing clickjacking protection

LOW

CVSS SCORE
6.5 CVSS:3.0/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N

METHOD	PATH
GET	https://crinava26.netlify.app/

DESCRIPTION

A **frameable response** occurs when one or multiple pages can be used on an iframe on any website. This allows the **clickjackin g** attack to be used.

Clickjacking is when an attacker a hidden iframe with multiple transparent or opaque layers above it, to trick a user into clicking on a button or link on the iframe when they were intending to click on the the top level page. Thus, the attacker is "hijacking" clic ks meant for the top level page and routing them to the iframe.

Using a similar technique, keystrokes can also be hijacked. With a carefully crafted combination of stylesheets, iframes, and text bo xes, a user can be led to believe they are typing in the password to their email or bank account, but are instead typing into an in visible frame controlled by the attacker.

EVIDENCE

Response headers, missing the X-Frame-Options header:

```
accept-ranges: bytes
age: 24
cache-control: public,max-age=0,must-revalidate
cache-status: "Netlify Edge"; hit
content-encoding: br
content-type: text/html; charset=UTF-8
date: Sat, 18 Jul 2026 12:13:23 GMT
etag: "f77462680fa9f1b82dd11b51c12a8f36-ssl-df"
server: Netlify
strict-transport-security: max-age=31536000; includeSubDomains; preload
vary: Accept-Encoding
x-nf-request-id: 01KXTJAGFC6ANTWGE0J5GS1K4Y
content-length: 662
```

REQUEST

```
GET / HTTP/2
accept: */*
accept-encoding: gzip, deflate, br
connection: keep-alive
user-agent: Mozilla/5.0 (compatible; +https://probely.com/sos)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36
ProbelySPDR/0.2.0
host: crinava26.netlify.app
```

RESPONSE

```
HTTP/2 200 OK
accept-ranges: bytes
age: 24
cache-control: public,max-age=0,must-revalidate
cache-status: "Netlify Edge"; hit
content-encoding: br
content-type: text/html; charset=UTF-8
date: Sat, 18 Jul 2026 12:13:23 GMT
etag: "f77462680fa9f1b82dd11b51c12a8f36-ssl-df"
```

```
server: Netlify
strict-transport-security: max-age=31536000; includeSubDomains; preload
vary: Accept-Encoding
x-nf-request-id: 01KXTJAGFC6ANTWGE0J5GS1K4Y
content-length: 662
<!doctype html>
<html lang="en" class="dark">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Crinava — The New Era of Cricket</title>
    <meta name="description" content="Crinava: Next-generation cricket
analytics, live match intelligence, and AI-powered predictions." />
    <script>
      window.process = { env: {} };
    </script>
    <!-- Celestial Organic Typography -->
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link href="https://fonts.googleapis.com/css2?family=JetBrains+Mono:wght@300
;400;500&family=Playfair+Display:wght@700;900&family=Inter:wght@400;500;600;700;
800;900&display=swap" rel="stylesheet" />
    <link href="https://fonts.googleapis.com/css2?family=Material+Symbols+Outlin
ed:wght,FILL@100..700,0..1&display=swap" rel="stylesheet" />
    <meta name='probely-verification'
content='c5ce0764-3330-42f5-a022-8a86effd23df' />
```

5 Browser content sniffing allowed

LOW

CVSS SCORE
4.7

CVSS:3.0/AV:N/AC:H/PR:N/UI:R/S:C/C:L/I:L/A:N

METHOD	PATH
GET	https://crinava26.netlify.app/

DESCRIPTION

The application allows browsers to try to mime-sniff the content-type of the responses. This means the browser may try to guess t he content-type by looking at the response content, and render it in way it was not intended to. This behavior may lead to the exe cution of malicious code, for instance, to explore an XSS vulnerability.

Applications should disable this behavior, forcing browsers to honor the content-type specified in the response. Without a specific c ontent-type set browsers will default to render the content as text, turning XSS payloads innocuous.

Disabling mime-sniffing should be seen as an extra layer of defense against XSS, and not as replacement of the recommended XS S prevention techniques.

EVIDENCE

Response headers, missing the X-Content-Type-Options header:

```
accept-ranges: bytes
age: 24
cache-control: public,max-age=0,must-revalidate
cache-status: "Netlify Edge"; hit
content-encoding: br
content-type: text/html; charset=UTF-8
date: Sat, 18 Jul 2026 12:13:23 GMT
etag: "f77462680fa9f1b82dd11b51c12a8f36-ssl-df"
server: Netlify
strict-transport-security: max-age=31536000; includeSubDomains; preload
vary: Accept-Encoding
x-nf-request-id: 01KXTJAGFC6ANTWGE0J5GS1K4Y
content-length: 662
```

REQUEST

```
GET / HTTP/2
accept: */*
accept-encoding: gzip, deflate, br
connection: keep-alive
user-agent: Mozilla/5.0 (compatible; +https://probely.com/sos)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36
ProbelySPDR/0.2.0
host: crinava26.netlify.app
```

RESPONSE

```
HTTP/2 200 OK
accept-ranges: bytes
age: 24
cache-control: public,max-age=0,must-revalidate
cache-status: "Netlify Edge"; hit
content-encoding: br
content-type: text/html; charset=UTF-8
date: Sat, 18 Jul 2026 12:13:23 GMT
etag: "f77462680fa9f1b82dd11b51c12a8f36-ssl-df"
server: Netlify
strict-transport-security: max-age=31536000; includeSubDomains; preload
```

```
vary: Accept-Encoding
x-nf-request-id: 01KXTJAGFC6ANTWGE0J5GS1K4Y
content-length: 662
<!doctype html>
<html lang="en" class="dark">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Crinava — The New Era of Cricket</title>
    <meta name="description" content="Crinava: Next-generation cricket
analytics, live match intelligence, and AI-powered predictions." />
    <script>
      window.process = { env: {} };
    </script>
    <!-- Celestial Organic Typography -->
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link href="https://fonts.googleapis.com/css2?family=JetBrains+Mono:wght@300
;400;500&family=Playfair+Display:wght@700;900&family=Inter:wght@400;500;600;700;
800;900&display=swap" rel="stylesheet" />
    <link href="https://fonts.googleapis.com/css2?family=Material+Symbols+Outlin
ed:wght,FILL@100..700,0..1&display=swap" rel="stylesheet" />
    <meta name='probely-verification'
content='c5ce0764-3330-42f5-a022-8a86effd23df' />
```

Glossary

Term	Definition
Vulnerability	A type of security weakness that might occur in applications (e.g. Broken Authentication, Information Disclosure). Some vulnerabilities take their name not from the weakness itself, but from the attack that exploits it (e.g. SQL Injection, XSS, etc.).
Findings	An instance of a Vulnerability that was found in an application.

Severity Legend

To each finding is attributed a severity which sums up its overall risk

The severity is a compound metric that encompasses the likelihood of the finding being found and exploited by an attacker, the skill required to exploit it, and the impact of such exploitation. A finding that is easy to find, easy to exploit and the exploitation has high impact, will have a greater severity.

Different findings of the same type could have a different severity: we consider multiple factors to increase or decrease it, such as if the application has an authenticated area or not.

The following table describes the different severities:

Severity	Description	Examples
CRITICAL	These findings require immediate attention and remediation due to their potentially devastating impact.	OS command injection SQL Injection
HIGH	These findings may have a direct impact on the application security, either clients or service owners, for instance, by granting the attacker access to sensitive information.	Reflected cross-site scripting Path Traversal
MEDIUM	Medium findings don't usually have an immediate impact alone, but combined with other findings, may lead to a successful compromise of the application.	Cross-site Request Forgery Unencrypted Communications
LOW	Findings where either the exploit is not trivial, the impact is low, or the finding cannot be exploited by itself.	Directory Listing Clickjacking

Category Descriptions

The following pages contain descriptions of each vulnerability. For each vulnerability you will find a section explaining its impact, causes and prevention methods. These descriptions are very generic, and whenever they are not enough to understand or fix a given finding, more information is provided for that finding in the Detailed Finding Descriptions section.

Weak cipher suites enabled

Description

The server supports weak cipher suites for SSL/TLS connections. These cipher suites are currently considered broken and, depending on the specific cipher suite, offer poor or no security at all. Thus defeating the purpose of using a secure communication channel in the first place.

Any connection to the server using a weak cipher suite is at risk of being eavesdropped and tampered with by an attacker that can intercept connections. This is more likely to occur to Wi-Fi clients.

Depending on the cipher suites used, a connection may be at an immediate risk of being intercepted.

Fix

To stop using weak cipher suites, you must configure your web server cipher suite list accordingly.

Ideally, as a general guideline, you should remove any cipher suite containing references to NULL, anonymous, export, DES, 3DES, RC4, and MD5 algorithms. Additionally, remove any cipher suite containing ciphers with less than 128 bit security. You should also remove any CBC ciphers, as CBC ciphers may be vulnerable to padding oracle attacks.

You should enable ECDHE and GCM cipher suites to ensure proper security. Please note that these modern ciphers are available in newer versions of TLS only. You will need to enable TLSv1.2 and above (for GCM cipher suites).

To achieve this, we propose a modern cipher suite, based on these recommendations:

TLS13-AES-256-GCM-SHA384:TLS13-CHACHA20-POLY1305-SHA256:TLS13-AES-128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-SHA384:ECDHE-RSA-AES256-SHA384:ECDHE-ECDSA-AES128-SHA256:ECDHE-RSA-AES128-SHA256

For most systems, changing TLS cipher suites, requires a change on the web server configuration file. Please refer to your web server documentation on how to do so.

Secure TLS protocol version 1.2 not supported

Description

TLS protocol version 1.2 is not enabled on your server. We strongly recommend that TLS 1.2 is enabled, as it is both safer and faster than previous versions and protocols.

One of the most important features of TLS 1.2 is that it supports AEAD ciphers (Authenticated Encryption with Associated Data). Some non-AEAD ciphers are vulnerable to a class of attacks called padding-oracles, which led to the publication of attacks such as Lucky Thirteen and POODLE, amongst others. Moreover, as of 2018, TLS 1.2 adoption is above 94%. By supporting AEAD ciphers, TLS 1.2 is much more robust against padding-oracle attacks.

Additionally, depending on CPU support, some AEAD ciphers are actually faster to execute than their older counterparts.

Fix

To fix this issue, you need to enable TLS 1.2+ in your webserver configuration.

The following guidelines should help you enable TLS 1.2+ with a strong cipher suite.

For the Apache server configuration, use this snippet as a guideline:

``` ...

```
SSLProtocol   -all +TLSv1.2 +TLSv1.3
SSLCipherSuite   ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-SHA384:ECDHE-RSA-AES256-
```
```

```
SHA384:ECDHE-ECDSA-AES128-SHA256:ECDHE-RSA-AES128-SHA256
SSLHonorCipherOrder   on
SSLCompression            off

...
```

For the Nginx server configuration, the following snippet may be used as a guideline:

```
server { listen 443 ssl;

...

ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers 'ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-SHA384:ECDHE-RSA-AES256-SHA384:ECDHE-ECDSA-AES128-SHA256:ECDHE-RSA-AES128-SHA256';
ssl_prefer_server_ciphers on;

....

}
```

We have written an in-depth article with more details here.

Referrer policy not defined

Description

The application does not prevent browsers from sending sensitive information to third party sites in the **referer** header.

Without a referrer policy, every time a user clicks a link that takes him to another origin (domain), the browser will add a **referer** header with the URL from which he is coming from. That URL may contain sensitive information, such as password recovery tokens or personal information, and it will be visible that other origin. For instance, if the user is at `example.com/password_recovery?unique_token=14f748d89d` and clicks a link to `example-analytics.com`, that origin will receive the complete password recovery URL in the headers and might be able to set the users password. The same happens for requests made automatically by the application, such as XHR ones.

Applications should set a secure referrer policy that prevents sensitive data from being sent to third party sites.

Fix

This problem can be fixed by sending the header **Referrer-Policy** with a secure and valid value. There are different values available, but not all are considered secure. Please note that this header only supports one directive at a time. The following list explains each one and it is ordered from the safest to the least safe:

- `no-referrer`: never send the header.
- `same-origin`: send the full URL to requests to the same origin (exact scheme + domain)
- `strict-origin`: send only the domain part of the URL, but sends nothing when downgrading to HTTP.
- `origin`: similar to **strict-origin** without downgrade restriction.
- `strict-origin-when-cross-origin`: send full URL within the same origin, but only the domain part when sending to another origin. It sends nothing when downgrading to HTTP.
- `origin-when-cross-origin`: similar to **strict-origin-when-cross-origin** without the downgrade restriction.

Insecure options: * `no-referrer-when-downgrade`: sends the full URL when the scheme does not change. It will send if both origins are, for instance, HTTP. * `unsafe-url`: always sent the full URL

A possible, safe option is `strict-origin`, so the header would look like this:

`Referrer-Policy: strict-origin`

It is normally easy to enable the header in the web server configuration file, but it can also be done at the application level.

Please note that the referrer header is written `referer`, with a single `r` but the referrer policy header is properly written, with `rr`: `Referrer-Policy`.

Missing Content Security Policy header

Description

The Content Security Policy (CSP) is an HTTP header through which site owners define a set of security rules that the browser must follow when rendering their site. The most common usage is to define a list of approved sources of content that the browser can load. This can be used to effectively mitigate Cross-Site Scripting (XSS) and Clickjacking attacks.

CSP is flexible enough for you to define from where the browser can load JavaScript, Stylesheets, images, or fonts, among other options. It can also be used in report mode only, a recommended approach before deploying strict rules in a live environment. However, please note that

report mode does not protect you, it just logs policy violations.

Fix

You can define a Content Security Policy by setting a header in your application. The header can look like this:

```
Content-Security-Policy: frame-ancestors 'none'; default-src 'self', script-src '*://*.example.com:*'
```

In this example, the **frame-ancestors** directive set to **'none'** indicates that the page cannot be placed inside a frame, not even by itself. The **default-src** defines the loading policy for all resources, in this case, they can be loaded from the current origin (protocol + domain + port). The example sets a more specific policy for scripts, through the **script-src**, restricting script loading to any subdomain of example.com.

The policy can be with different directives, and there are other less strict options for the directives above.

Missing clickjacking protection

Description

A **frameable response** occurs when one or multiple pages can be used on an iframe on any website. This allows the **clickjacking** attack to be used.

Clickjacking is when an attacker a hidden iframe with multiple transparent or opaque layers above it, to trick a user into clicking on a button or link on the iframe when they were intending to click on the the top level page. Thus, the attacker is "hijacking" clicks meant for the top level page and routing them to the iframe.

Using a similar technique, keystrokes can also be hijacked. With a carefully crafted combination of stylesheets, iframes, and text boxes, a user can be led to believe they are typing in the password to their email or bank account, but are instead typing into an invisible frame controlled by the attacker.

Fix

The recommended way to prevent clickjacking is to send a header that instructs the browser to not allow arbitrary framing, typically from other domains.

The current recommendation is to use the Content-Security-Policy HTTP header (CSP) with a **frame-ancestors** directive. This header obsoletes the X-Frame-Options HTTP header.

To use CSP you need the following header:

```
Content-Security-Policy: frame-ancestors 'none'
```

The header might contain more directives, and there are other less strict options for the **frame-ancestors** directive.

If you want to use X-Frame-Options, send the proper HTTP header, with one of the following directives:

```
X-Frame-Options: DENY
X-Frame-Options: SAMEORIGIN
```

A third directive, **ALLOW-FROM** is no longer supported by modern browsers.

If you specify **DENY**, all attempts to load the page in a frame will fail. **SAMEORIGIN** will allow the page to be loaded in the site including it in a frame is the same as the one serving the page.

The most common option is **DENY** when there is no need to load your pages on some other site.

Browser content sniffing allowed

Description

The application allows browsers to try to mime-sniff the content-type of the responses. This means the browser may try to guess the content-type by looking at the response content, and render it in way it was not intended to. This behavior may lead to the execution of malicious code, for instance, to explore an XSS vulnerability.

Applications should disable this behavior, forcing browsers to honor the content-type specified in the response. Without a specific content-type set browsers will default to render the content as text, turning XSS payloads innocuous.

Disabling mime-sniffing should be seen as an extra layer of defense against XSS, and not as replacement of the recommended XSS prevention techniques.

Fix

This problem can be fixed by sending the header **X-Content-Type-Options** with value **nosniff**, to force browsers to disable the content-type guessing (the sniffing).

The header should look this:

```
X-Content-Type-Options: nosniff
```

It is normally easy to enable the header in the web server configuration file, but it can also be done at application level.